

■ ASVANNA (■■■■■■■■■■)

The Zero-Waste Marketplace: Guided by Real-Time Data from Seed to Harvest Distribution

Academic Institution:	Division of Information Technology, Institute of Technology, University of Moratuwa (ITUM)
Qualification:	National Diploma in Information Technology (NDIT)
Academic Year:	Final Year Project 2025 / 2026
Project Supervisor:	Mrs. Uthpala Athukorala
Document Purpose:	Complete System Architecture, File-by-File Code Documentation, Member Allocation & QA Test Manual

Group 15 — Team Members & Student Registration Numbers:

#	Student Name	Student ID	Assigned Core Engineering Module
1	W. N. A. Wedikkara	23IT0544	System Architecture, Auth/RBAC, DB & Risk Engine
2	K. A. H. I. Lakshitha	23IT0503	Smart Crop Recommendation & Agro-Scoring Engine
3	G. W. T. Jayampathi	23IT0487	Divisional Officer Portal, Proxy Entry & Broadcast Alerts
4	R. R. L. Geeganage	23IT0476	Farmer Mobile App (Flutter), GPS Logging & Offline Queue
5	K. H. M. Dewanga	23IT0467	5 km Geo-Fenced Zero-Waste Surplus Marketplace

This document provides a dual-audience manual: non-technical agricultural stakeholders (farmers, agrarian officers) and technical academic evaluators (software engineers, examiners). Explanations are presented in English with complementary Singlish & Sinhala insights.

1. Executive Summary & Plain-Language Overview

ASVANNA (ASVANA) — Master System Manual & Technical Documentation

Sri Lanka's upcountry vegetable farmers (Bandarawela, Nuwara Eliya, Badulla) suffer massive recurring losses (estimated at 30%–40% post-harvest waste, exceeding Rs. 180 Billion annually). This happens because of **'Trend Planting'** (සම මගින්) — when carrot or leeks prices rise at Dambulla or Manning Market, hundreds of farmers simultaneously cultivate the same crop. Months later, thousands of tons flood the market at the exact same week, causing disastrous price crashes and produce dumping.

Singlish / Sinhala Explanation (Non-IT Person Guide):

'Govi mahathwaru market eke ada thiyena ganan balala ekama boga wargaya (e.g. Leeks, Cabbage) ekawara wawanawa. Ethakota mas 3kin ekawara aswanna labunama market eka pirila mila bahinawa. ASVANNA kiyanne meka nawathwana digital platform ekak. Farmer boga wawanna kalin app eken log kalama, system eka national market demand eka (CROPIX) ekka compare karala warning ekak denawa. Saturation eka 85% wadi nam wena labadayaka boga (Beetroot, Knol Khol) recommend karanawa. Ithuru wena aswanna 5km athule buyerslata kelinma wikunanna marketplace ekakuth meke thiyana.'

Key Ecosystem Pillars:

Pillar 1: Regional Cultivation Monitoring	Real-time GPS plotting of active planting plots across agrarian divisions.
Pillar 2: Predictive Risk Engine	Calculates supply density against national demand (CROPIX) to give 3-tier alerts: Safe (<70%), At-Risk (70–85%), Over-Planted (>85%).
Pillar 3: Smart Recommendations	4-factor composite scoring: Market Gap (35%), Soil Suitability (25%), Weather (20%), Price Trend (20%).
Pillar 4: Zero-Waste Marketplace	Geo-fenced 5 km direct trade connecting farmers with local hoteliers, caterers, and bulk buyers for surplus produce.
Pillar 5: Digital Inclusivity (Proxy Entry)	Divisional Officers enter cultivation records on behalf of farmers who lack smartphones, ensuring 100% regional map completeness.

2. Group 15 — Individual Member Breakdown & Evaluation Guide

Each team member is responsible for a dedicated, modular tier of the ASVANNA ecosystem. Evaluators can test each student's specific contribution using the steps below:

Member 1: W. N. A. Wedikkara (23IT0544)

Assigned Responsibility: System Architecture, PostgreSQL Relational Schema, JWT Role-Based Access Control (RBAC), and Mathematical Predictive Risk Engine.

Files Authored: backend/src/services/riskEngineService.js, backend/src/controllers/riskEngineController.js, backend/src/database/schema.sql, backend/src/middlewares/authMiddleware.js, backend/src/config/database.js

How Evaluators Can Test Member 1's Work:

1. Run `npm run migrate` & `npm run seed` in backend/ to verify DB relational schemas, indices, and seed data.
 2. Send a GET request to `http://localhost:5000/api/v1/risk/crop/1` to test the Leeks supply vs CROPIX demand calculation.
 3. Verify that JWT tokens enforce 24-hour expiration for Farmers and 8-hour expiration for Officers.
-

Member 2: K. A. H. I. Lakshitha (23IT0503)

Assigned Responsibility: Smart Crop Recommendation Engine, Multi-factor Weighted Scoring Algorithm (Market Gap 35%, Soil 25%, Weather 20%, Historical Price 20%), and Agronomic Suitability Models.

Files Authored: backend/src/services/recommendationService.js, backend/src/controllers/recommendationController.js, backend/src/routes/recommendationRoutes.js, frontend/src/pages/RiskAnalytics.jsx

How Evaluators Can Test Member 2's Work:

1. Query `http://localhost:5000/api/v1/recommendations?district=Badulla&cropId=1`.
 2. Verify that when Leeks (cropId 1) is at risk, candidate alternative crops like Beetroot (Score 92%) and Knol Khol (Score 88%) are ranked descending by composite score with Sinhala/Tamil/English rationales.
-

Member 3: G. W. T. Jayampathi (23IT0487)

Assigned Responsibility: Divisional Officer (DO) Web Portal, Digital Inclusivity Proxy Data Entry, Regional Cultivation Heatmap & Broadcast Warning System with FCM Push & SMS Fallback.

Files Authored: frontend/src/pages/Dashboard.jsx, frontend/src/pages/RegionalMonitoring.jsx, frontend/src/components/ProxyDataModal.jsx, frontend/src/components/BroadcastModal.jsx, backend/src/services/notificationService.js, frontend/src/locales/*

How Evaluators Can Test Member 3's Work:

1. Open `http://localhost:3000` and login using Officer credentials (0771234567 / asvanna123).
 2. Click '+ Proxy Data Entry' and submit a record on behalf of an offline farmer.
 3. Click 'Broadcast Warnings' to issue an emergency push notification to registered regional farmers.
 4. Toggle between English, Sinhala, and Tamil languages in the navbar.
-

Member 4: R. R. L. Geeganage (23IT0476)

Assigned Responsibility: Cross-Platform Flutter Mobile Application for Farmers, GPS-Tagged Field Logging, Offline SQLite/SharedPreferences Caching, and Auto-Sync Engine.

Files Authored: mobile/lib/features/home/farmer_home_screen.dart,
mobile/lib/features/planting/log_planting_screen.dart,
mobile/lib/core/services/location_service.dart,
mobile/lib/core/services/offline_storage_service.dart, mobile/lib/l10n/*

How Evaluators Can Test Member 4's Work:

1. Run flutter run inside mobile/ directory.
 2. Click 'Log Planting', verify that current GPS coordinates are automatically fetched.
 3. Disconnect network on the emulator, submit a record to verify offline queuing, then reconnect to observe auto-sync.
-

Member 5: K. H. M. Dewanga (23IT0467)

Assigned Responsibility: Geo-Fenced Zero-Waste Surplus Marketplace (Phase 2), Haversine 5 km Proximity Radius Matching, Direct Buyer-Farmer Negotiation & 30-Minute Counter-Offer State Machine.

Files Authored: backend/src/services/geofencingService.js,
backend/src/services/marketplaceService.js, backend/src/controllers/marketplaceController.js,
frontend/src/pages/MarketplaceSurplus.jsx, mobile/lib/features/marketplace/*

How Evaluators Can Test Member 5's Work:

1. Send a POST request to /api/v1/marketplace/list to publish a surplus crop batch.
2. Send a GET to /api/v1/marketplace/search-nearby?lat=6.8258&lng=80.9982&radius=_km=5 to verify listings within 5 km.
3. Place a buyer order to verify the 30-minute deadline negotiation timer.

3. Exhaustive File-by-File Codebase Documentation

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

This section documents every single file in the repository, explaining its purpose, dependencies, line logic, and required configuration.

■ docker-compose.yml — Root Orchestration

Primary Purpose:	Spins up PostgreSQL 15, Node.js REST API, and React Web Admin simultaneously.
Connected Files / Dependencies:	Docker Engine
Required Configurations (.env):	POSTGRES_DB, PORT
Code Logic Breakdown:	Docker compose service blocks defining container dependencies and volume mounts.
Verification / Test Command:	<code>docker-compose up --build</code>

■ .github/workflows/ci.yml — Continuous Integration

Primary Purpose:	GitHub Actions CI pipeline running linting, tests, and build checks on pull requests.
Connected Files / Dependencies:	GitHub Runners
Required Configurations (.env):	NODE_ENV=test
Code Logic Breakdown:	Defines backend-checks and frontend-checks workflows.
Verification / Test Command:	<code>git push origin main</code>

■ setup-individual-repos.sh — Multi-repo Initializer

Primary Purpose:	Shell script to split the monorepo into 3 standalone Git repositories (backend, frontend, mobile).
Connected Files / Dependencies:	Bash, Git
Required Configurations (.env):	None
Code Logic Breakdown:	Iterates through subdirectories and runs <code>git init -b main</code> .
Verification / Test Command:	<code>./setup-individual-repos.sh</code>

■ backend/src/server.js — Backend Entry Point

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Starts the HTTP server on configured port and listens for requests.
Connected Files / Dependencies:	app.js, config.js
Required Configurations (.env):	PORT=5000
Code Logic Breakdown:	Initializes app.listen and handles graceful SIGTERM termination.
Verification / Test Command:	npm start

■ backend/src/app.js — Express App Setup

Primary Purpose:	Configures security middleware (helmet, cors), routes, and global error handling.
Connected Files / Dependencies:	express, routes
Required Configurations (.env):	None
Code Logic Breakdown:	Mounts /api/v1 endpoints and global 404/error handlers.
Verification / Test Command:	npm run dev

■ backend/src/config/config.js — Configuration Module

Primary Purpose:	Centralizes all environment variables (.env) with safe fallback defaults.
Connected Files / Dependencies:	dotenv
Required Configurations (.env):	DB_*, JWT_*, RISK_*
Code Logic Breakdown:	Parses process.env into structured JS objects.
Verification / Test Command:	node -e 'require("./src/config/config")'

■ backend/src/config/database.js — PostgreSQL Pool

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Creates a reusable PostgreSQL connection pool using 'pg'.
Connected Files / Dependencies:	pg, config.js
Required Configurations (.env):	DB_HOST, DB_NAME
Code Logic Breakdown:	Exports pool.query method with automatic error logging.
Verification / Test Command:	node src/config/database.js

■ backend/src/config/firebase.js — Firebase Admin SDK

Primary Purpose:	Initializes Firebase Cloud Messaging (FCM) & Realtime Database with fallback.
Connected Files / Dependencies:	firebase-admin
Required Configurations (.env):	FIREBASE_PRIVATE_KEY
Code Logic Breakdown:	Initializes admin SDK if credentials present, otherwise falls back.
Verification / Test Command:	node src/config/firebase.js

■ backend/src/database/schema.sql — PostgreSQL DDL

Primary Purpose:	Defines 10 core tables: users, crops, planting_records, cropix_benchmarks, risk, etc.
Connected Files / Dependencies:	PostgreSQL 12+
Required Configurations (.env):	asvanna_db
Code Logic Breakdown:	SQL DDL with UUID, foreign keys, and indexes.
Verification / Test Command:	psql -d asvanna_db -f schema.sql

■ backend/src/database/migrate.js — Migration Runner

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Executes schema.sql to initialize or update PostgreSQL tables.
Connected Files / Dependencies:	fs, database.js
Required Configurations (.env):	DB credentials
Code Logic Breakdown:	Reads schema.sql and sends raw SQL via pool.query.
Verification / Test Command:	npm run migrate

■ backend/src/database/seed.js — Pilot Seed Script

Primary Purpose:	Seeds 8 master upcountry crops, demo users, and CROPIX demand benchmarks.
Connected Files / Dependencies:	bcryptjs, database.js
Required Configurations (.env):	DB credentials
Code Logic Breakdown:	Hashes default passwords and populates master data.
Verification / Test Command:	npm run seed

■ backend/src/middlewares/authMiddleware.js — JWT & RBAC Middleware

Primary Purpose:	Validates Bearer token and checks user role permissions.
Connected Files / Dependencies:	jsonwebtoken, config.js
Required Configurations (.env):	JWT_SECRET
Code Logic Breakdown:	authenticate() verifies token, authorizeRoles() checks user.role.
Verification / Test Command:	curl with Authorization: Bearer

■ backend/src/middlewares/errorHandler.js — Global Error Handler

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Catches all uncaught server exceptions and returns standard JSON response.
Connected Files / Dependencies:	apiResponse.js
Required Configurations (.env):	NODE_ENV
Code Logic Breakdown:	Extracts err.statusCode, formats message, hides stack in production.
Verification / Test Command:	Trigger 404/500 endpoint

■ backend/src/middlewares/validationMiddleware.js — Request Validator

Primary Purpose:	Validates input schemas using express-validator.
Connected Files / Dependencies:	express-validator
Required Configurations (.env):	None
Code Logic Breakdown:	Extracts validation errors and returns 400 Bad Request if invalid.
Verification / Test Command:	Send empty POST payload

■ backend/src/utils/haversine.js — Haversine Distance Calculator

Primary Purpose:	Calculates great-circle distance (km) between two GPS points.
Connected Files / Dependencies:	Math library
Required Configurations (.env):	None
Code Logic Breakdown:	Applies trigonometric Haversine formula on lat/lon coordinates.
Verification / Test Command:	node backend/test/test_all_endpoints.js

■ backend/src/utils/apiResponse.js — Standard API Formatter

Primary Purpose:	Formats standard JSON responses: { success, message, data, timestamp }.
Connected Files / Dependencies:	None
Required Configurations (.env):	None
Code Logic Breakdown:	Static success() and error() helper functions.
Verification / Test Command:	Imported in controllers

■ backend/src/services/riskEngineService.js — Predictive Risk Engine

Primary Purpose:	Calculates regional supply vs CROPIX demand and evaluates 3-tier risk.
Connected Files / Dependencies:	database.js, config.js
Required Configurations (.env):	RISK_SAFE_THRESHOLD
Code Logic Breakdown:	Aggregates planted acres * avg yield / target demand.
Verification / Test Command:	curl /api/v1/risk/crop/1

■ backend/src/services/recommendationService.js — Smart Crop Recommender

Primary Purpose:	Ranks alternative crops using 4-factor composite scoring formula.
Connected Files / Dependencies:	riskEngineService.js
Required Configurations (.env):	None
Code Logic Breakdown:	Calculates market gap, soil, weather, price scores and sorts descending.
Verification / Test Command:	curl /api/v1/recommendations

■ backend/src/services/geofencingService.js — Geofencing Engine

Primary Purpose:	Filters marketplace listings within 5 km - 20 km radius.
Connected Files / Dependencies:	haversine.js
Required Configurations (.env):	GEOFENCE_RADIUS
Code Logic Breakdown:	Maps items with distanceKm and filters by radius limit.
Verification / Test Command:	curl /api/v1/marketplace/search-nearby

■ backend/src/services/notificationService.js — Notification & SMS Service

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Sends FCM push notifications with SMS fallback for offline farmers.
Connected Files / Dependencies:	firebase.js, database.js
Required Configurations (.env):	SMS_GATEWAY_URL
Code Logic Breakdown:	Sends multicast push via FCM and logs SMS fallback dispatch.
Verification / Test Command:	Trigger broadcast warning

■ backend/src/services/marketplaceService.js — Realtime Sync Service

Primary Purpose:	Synchronizes active produce listings with Firebase Realtime Database.
Connected Files / Dependencies:	firebase.js
Required Configurations (.env):	FIREBASE_DATABASE_URL
Code Logic Breakdown:	Updates /marketplace_listings ref in Firebase RTDB.
Verification / Test Command:	Publish new surplus listing

■ backend/src/controllers/authController.js — Authentication Logic

Primary Purpose:	Handles User Registration, Login, Password Hashing, Profile, and FCM token update.
Connected Files / Dependencies:	bcryptjs, jwt
Required Configurations (.env):	JWT_SECRET
Code Logic Breakdown:	Registers user, verifies bcrypt hash, issues signed JWT.
Verification / Test Command:	POST /api/v1/auth/login

■ backend/src/controllers/plantingController.js — Planting Controller

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Logs GPS planting records (Farmer/Officer proxy) and triggers risk recalculation.
Connected Files / Dependencies:	database.js, riskEngine
Required Configurations (.env):	None
Code Logic Breakdown:	Inserts planting_records and returns updated risk assessment.
Verification / Test Command:	POST /api/v1/planting/log

■ backend/src/controllers/riskEngineController.js — Risk API Controller

Primary Purpose:	Exposes endpoints for individual crop risk and regional district summaries.
Connected Files / Dependencies:	riskEngineService.js
Required Configurations (.env):	None
Code Logic Breakdown:	Calls RiskEngineService and sends formatted response.
Verification / Test Command:	GET /api/v1/risk/regional-summary

■ backend/src/controllers/recommendationController.js — Recommendation API

Primary Purpose:	Exposes smart crop recommendation endpoint.
Connected Files / Dependencies:	recommendationService.js
Required Configurations (.env):	None
Code Logic Breakdown:	Returns top 5 ranked alternative crops.
Verification / Test Command:	GET /api/v1/recommendations

■ backend/src/controllers/marketplaceController.js — Marketplace Controller

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Handles surplus listing creation, proximity search, and direct order offers.
Connected Files / Dependencies:	geofencingService.js
Required Configurations (.env):	None
Code Logic Breakdown:	Manages listings, geofiltering, and order response deadlines.
Verification / Test Command:	POST /api/v1/marketplace/list

■ backend/src/controllers/officerController.js — Officer Admin Controller

Primary Purpose:	Manages farmer directory and proxy registrations.
Connected Files / Dependencies:	database.js, bcryptjs
Required Configurations (.env):	None
Code Logic Breakdown:	Queries farmers list and creates proxy farmer accounts.
Verification / Test Command:	GET /api/v1/officer/farmers

■ backend/src/controllers/broadcastController.js — Broadcast Controller

Primary Purpose:	Creates officer warning announcements and triggers multi-channel alerts.
Connected Files / Dependencies:	notificationService.js
Required Configurations (.env):	None
Code Logic Breakdown:	Inserts broadcast_warnings and notifies farmers.
Verification / Test Command:	POST /api/v1/broadcasts

■ frontend/src/App.jsx — React Router & Root

Primary Purpose:	Main router configuring protected routes and layout hierarchy.
Connected Files / Dependencies:	react-router-dom, contexts
Required Configurations (.env):	None
Code Logic Breakdown:	Conditionally renders Sidebar/Navbar or Login based on auth state.
Verification / Test Command:	npm run dev

■ frontend/src/context/AuthContext.jsx — Auth State Provider

Primary Purpose:	Stores user token and profile state across the React app.
Connected Files / Dependencies:	services/api.js
Required Configurations (.env):	asvanna_token
Code Logic Breakdown:	Provides user, login, logout, and token methods.
Verification / Test Command:	Login / Logout actions

■ frontend/src/context/LanguageContext.jsx — Language State Provider

Primary Purpose:	Provides trilingual switching across EN, SI, and TA.
Connected Files / Dependencies:	locales/*.json
Required Configurations (.env):	asvanna_lang
Code Logic Breakdown:	Provides t(key) translation function.
Verification / Test Command:	Switch language in navbar

■ frontend/src/pages/Dashboard.jsx — Main Officer Dashboard

Primary Purpose:	Shows KPI summary cards, crop saturation progress bars, and quick modals.
Connected Files / Dependencies:	StatCard, Modals
Required Configurations (.env):	None
Code Logic Breakdown:	Fetches /risk/regional-summary and displays stats.
Verification / Test Command:	View http://localhost:3000

■ frontend/src/pages/RegionalMonitoring.jsx — Cultivation Map Page

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Displays GPS planting plots across Bandarawela sub-divisions.
Connected Files / Dependencies:	services/api.js
Required Configurations (.env):	None
Code Logic Breakdown:	Renders interactive grid and filterable plots list.
Verification / Test Command:	View /monitoring

■ frontend/src/pages/RiskAnalytics.jsx — Deep Risk Analytics

Primary Purpose:	Visualizes saturation thresholds and recommended crop alternatives.
Connected Files / Dependencies:	services/api.js
Required Configurations (.env):	None
Code Logic Breakdown:	Renders risk cards and 4-factor composite scores.
Verification / Test Command:	View /risk-analytics

■ frontend/src/pages/FarmerDirectory.jsx — Farmer Directory Page

Primary Purpose:	Searchable list of registered farmers (smartphone & proxy profiles).
Connected Files / Dependencies:	services/api.js
Required Configurations (.env):	None
Code Logic Breakdown:	Filters farmers by name, phone, or NIC.
Verification / Test Command:	View /farmers

■ frontend/src/pages/Broadcasts.jsx — Broadcast Warnings Page

Primary Purpose:	Displays warning history and lets officers dispatch new alerts.
Connected Files / Dependencies:	BroadcastModal.jsx
Required Configurations (.env):	None
Code Logic Breakdown:	Fetches /broadcasts and displays severity tags.
Verification / Test Command:	View /broadcasts

■ frontend/src/pages/MarketplaceSurplus.jsx — Surplus Marketplace View

Primary Purpose:	Displays active 5 km surplus produce batches for local trade.
Connected Files / Dependencies:	None
Required Configurations (.env):	None
Code Logic Breakdown:	Lists available surplus produce cards with direct buy CTA.
Verification / Test Command:	View /marketplace

■ mobile/lib/main.dart — Flutter Entry Point

Primary Purpose:	Initializes Flutter engine, localization delegates, and sets initial route.
Connected Files / Dependencies:	flutter_localizations
Required Configurations (.env):	None
Code Logic Breakdown:	Sets up MaterialApp, ThemeData, and FarmerHomeScreen.
Verification / Test Command:	flutter run

■ mobile/lib/core/constants/app_colors.dart — Mobile Theme Colors

Primary Purpose:	Centralizes agricultural emerald greens and risk color codes.
Connected Files / Dependencies:	flutter/material.dart
Required Configurations (.env):	None
Code Logic Breakdown:	Defines primary, accent, riskSafe, riskWarning, riskOverPlanted.
Verification / Test Command:	Referenced across UI

■ mobile/lib/core/services/location_service.dart — GPS Location Service

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Primary Purpose:	Fetches device current GPS latitude/longitude using Geolocator.
Connected Files / Dependencies:	geolocator
Required Configurations (.env):	GPS Permission
Code Logic Breakdown:	Queries Geolocator.getCurrentPosition() with Bandarawela fallback.
Verification / Test Command:	Test on GPS device

■ mobile/lib/core/services/offline_storage_service.dart — Offline Queue Service

Primary Purpose:	Stores offline planting records locally in SharedPreferences.
Connected Files / Dependencies:	shared_preferences
Required Configurations (.env):	None
Code Logic Breakdown:	Saves JSON string queue and provides clear/sync methods.
Verification / Test Command:	Test in Airplane mode

■ mobile/lib/features/home/farmer_home_screen.dart — Farmer Home Screen

Primary Purpose:	Main mobile UI showing weather, over-planting alerts, and active plantings.
Connected Files / Dependencies:	planting, recs, market
Required Configurations (.env):	None
Code Logic Breakdown:	Bottom navigation tabs for Home, Recommendations, and Marketplace.
Verification / Test Command:	View on mobile emulator

■ mobile/lib/features/planting/log_planting_screen.dart — Planting Log Screen

Primary Purpose:	Farmer data entry form with auto GPS location tag and crop selection.
Connected Files / Dependencies:	location_service.dart
Required Configurations (.env):	None
Code Logic Breakdown:	Submits planting payload to API or offline queue.
Verification / Test Command:	Tap 'Log Planting' button

■ mobile/lib/features/recommendations/smart_crop_recommendation_screen.dart —**Smart Recs Mobile View**

Primary Purpose:	Displays ranked alternative crops with market gap and suitability score.
Connected Files / Dependencies:	constants/app_colors.dart
Required Configurations (.env):	None
Code Logic Breakdown:	Renders recommendation cards with Sinhala rationale.
Verification / Test Command:	Tap 'Recommendations' tab

■ mobile/lib/features/marketplace/marketplace_feed_screen.dart — Marketplace Feed Screen

Primary Purpose:	Lists surplus produce within 5 km radius for buyers and farmers.
Connected Files / Dependencies:	list_surplus_screen.dart
Required Configurations (.env):	None
Code Logic Breakdown:	Displays produce listings with asking price and distance.
Verification / Test Command:	Tap 'Marketplace' tab

4. Comprehensive Testing & Verification Manual

ASVANNA (■■■■■■■■■■) — Master System Manual & Technical Documentation

Step-by-step instructions to test and verify every tier of the ASVANNA platform:

Step 1: Automated Unit & Math Verification

Command: `node backend/test/test_all_endpoints.js`

Description: Executes 4 automated unit tests verifying Haversine calculations, 3-tier risk logic, and composite recommendation weight sums.

Step 2: Database Migration & Seeding

Command: `cd backend && npm run migrate && npm run seed`

Description: Initializes PostgreSQL schema (10 tables) and inserts 8 upcountry crops, 5 demo accounts, and monthly CROPIX demand quotas.

Step 3: Backend REST API Launch

Command: `cd backend && npm run dev`

Description: Starts server on `http://localhost:5000`. Verify health endpoint at `http://localhost:5000/health`.

Step 4: Web Admin Portal Launch

Command: `cd frontend && npm run dev`

Description: Starts React dashboard on `http://localhost:3000`. Login with Officer credentials (0771234567 / asvanna123).

Step 5: Mobile App Launch

Command: `cd mobile && flutter pub get && flutter run`

Description: Launches the mobile app on an Android emulator or device for Farmer data logging and Surplus marketplace browsing.

Step 6: Docker Compose Launch

Command: `docker-compose up --build -d`

Description: Spins up the full multi-tier ecosystem (PostgreSQL, Backend API, Web Admin) in isolated Docker containers.

*End of Master System Documentation — Prepared for ITUM University of Moratuwa National Diploma in IT
Final Year Evaluation 2026.*